



**POLITÉCNICO
ESTELLA**

ASIGNATURA: ACCESO A DATOS

**GRADO SUPERIOR EN DESARROLLO DE
APLICACIONES MULTIPLATAFORMA**

AD06 TAREA

Presentado por: Eugen Moga

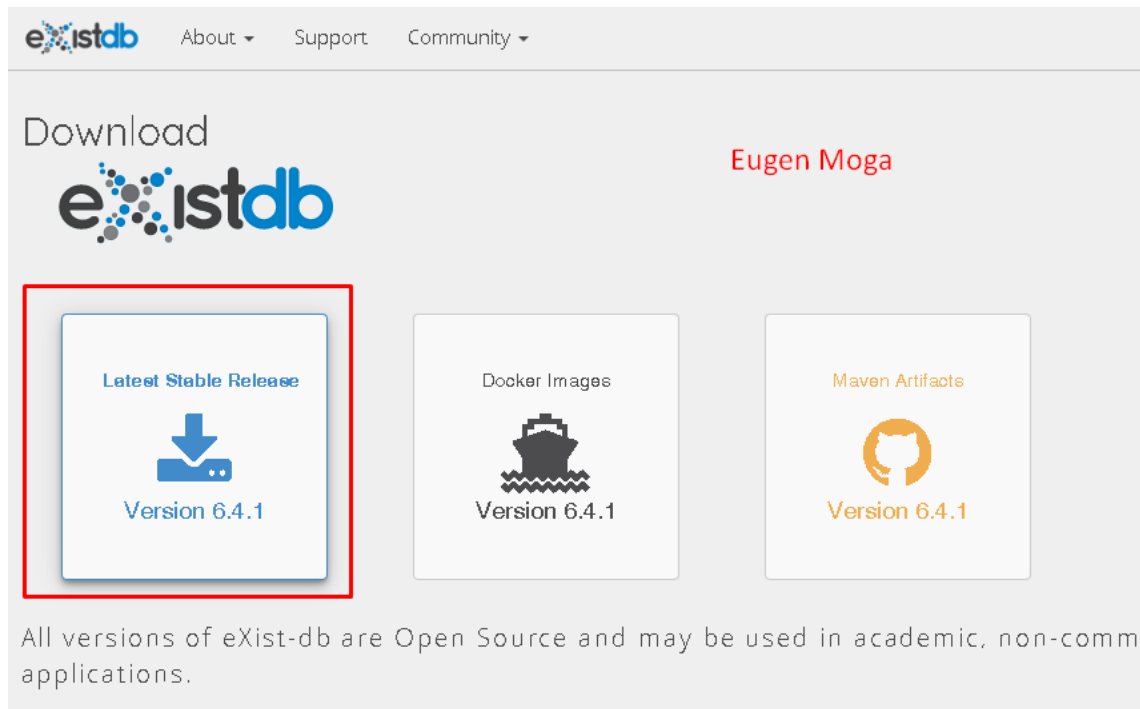
Fecha: 11/03/2026

Índice

Contexto	2
Ejercicio 1 XPATH (universidad.xml)	9
Ejercicio 2 XQUERY (libros.xml)	14
EJERCICIO 3. (utiliza eXist en un programa JAVA)	21
Clase BaseDatos_eXist.java	22
Clase AD06_ejercicio3.java	26
Conectores	29
Funcionamiento del programa	30

Contexto

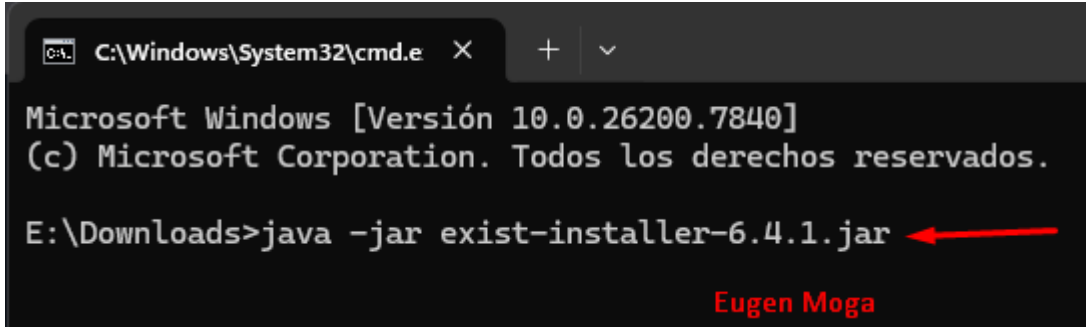
Antes de empezar con los ejercicios procedo a documentar la instalación de eXist-db para ello lo primero voy a la web oficial de [eXist](#) en la parte de Download y descargo la version 6.4.1



Eso me lleva a la web de github donde esta todos los descargables selecciono el installer.jar para descargar



Voy a la carpeta de descargas abro el cmd y ejecuto el .jar con el comando.



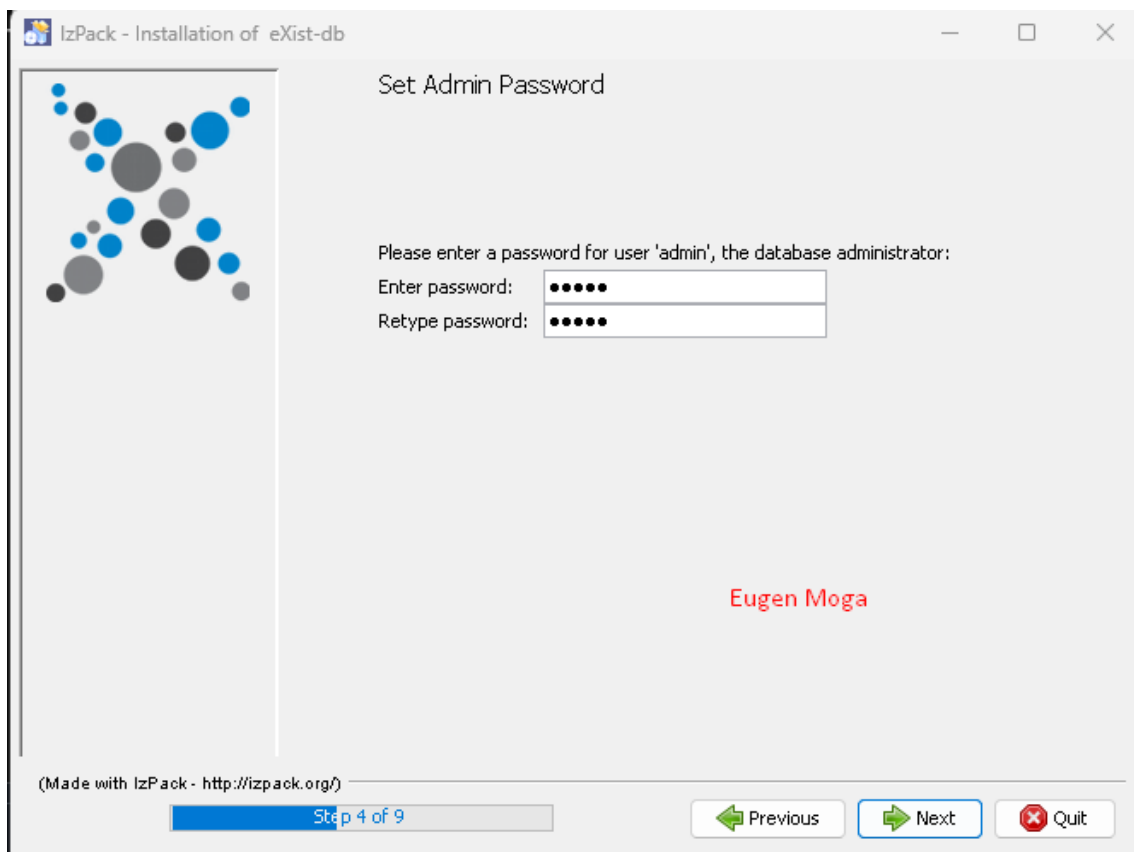
```
C:\Windows\System32\cmd.e X + v
Microsoft Windows [Versión 10.0.26200.7840]
(c) Microsoft Corporation. Todos los derechos reservados.

E:\Downloads>java -jar exist-installer-6.4.1.jar
```

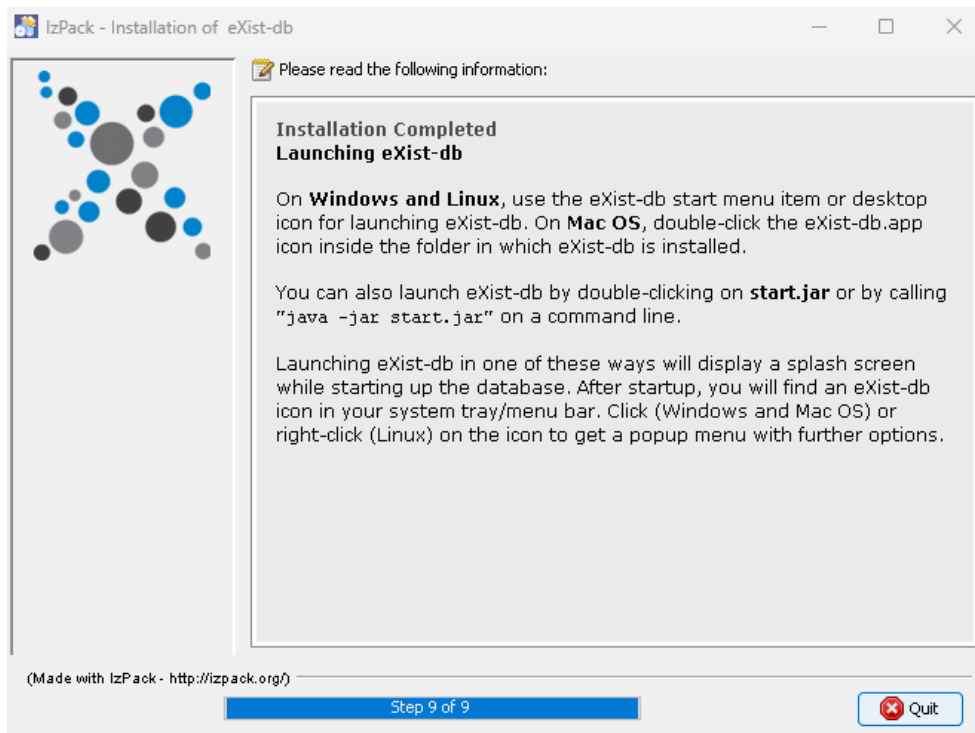
Eugen Moga

Sigo los pasos del instalador, dejo la ruta de instalación por defecto C:\eXist-db, también dejo por defecto el directorio de los datos C:\eXist-db\data y le pulso en siguiente.

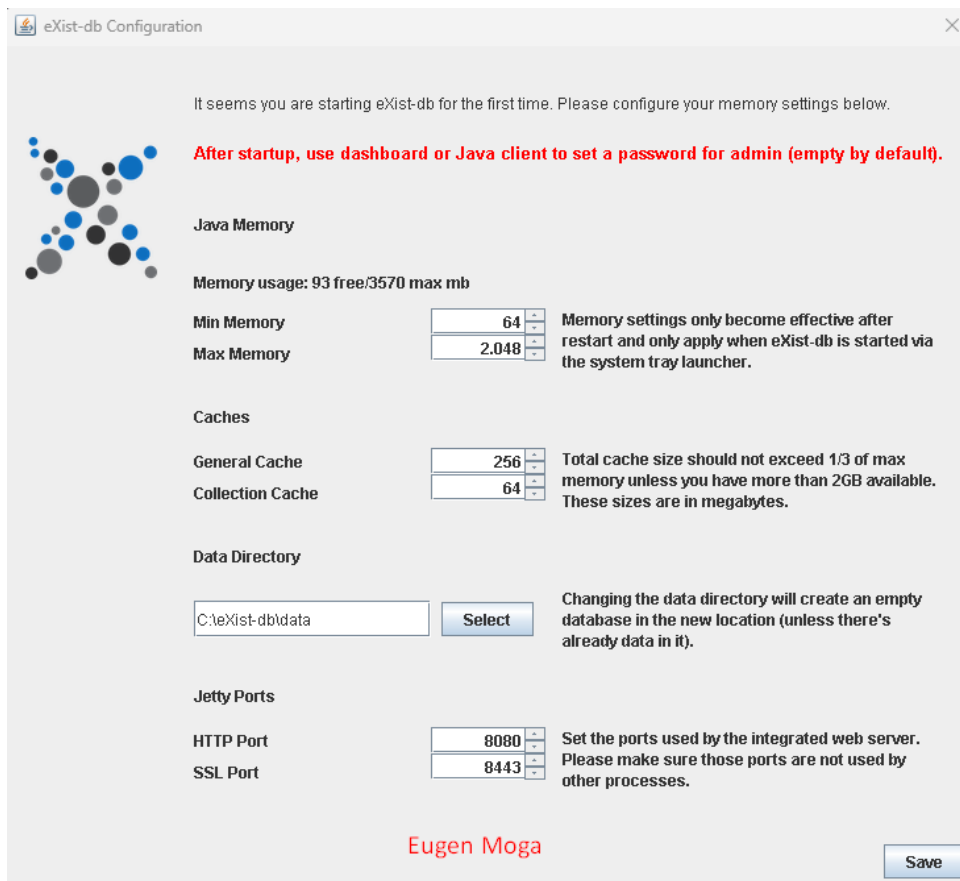
Me pide configurar una contraseña para el usuario admin y le pongo de contraseña admin.



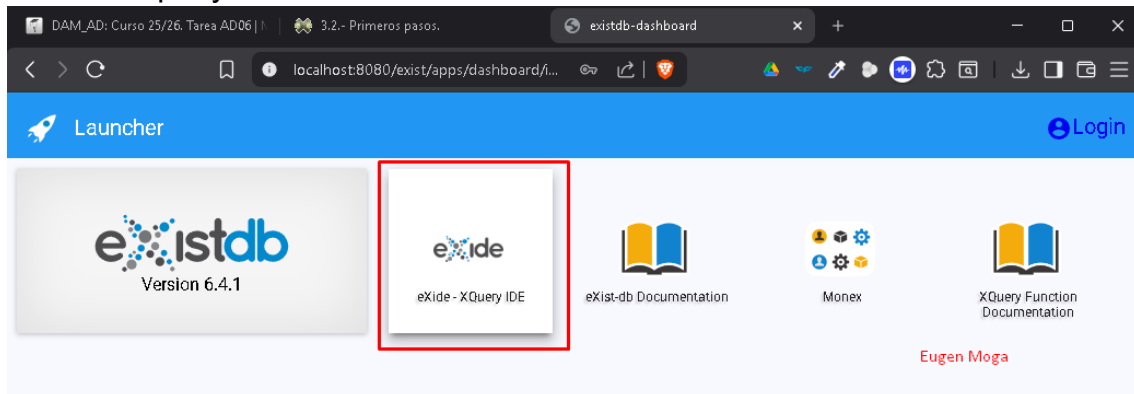
Al finalizar los pasos del instalador me sale que se ha completado la instalación y le pulso en Quit



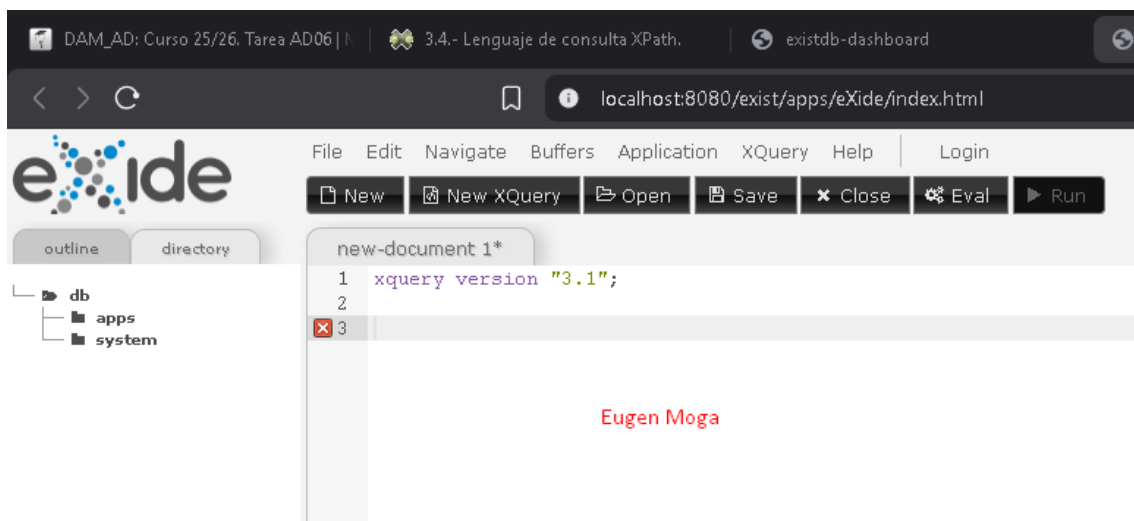
Ejecuto la aplicación y me sale la pantalla de configuración de todo por defecto, memoria, puertos y le pulso en el botón Save para guardar.



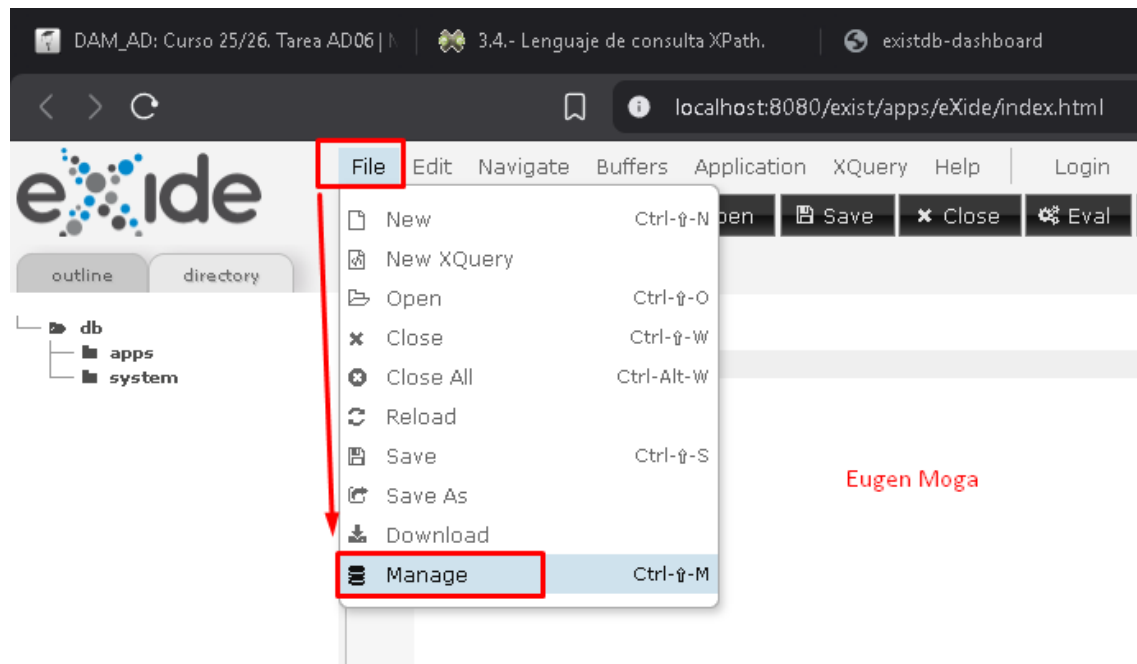
Ahora para acceder al servidor de eXist abro el navegador y pongo en la barra de navegación <http://localhost:8080> puedo ver la interfaz que ofrece la aplicación. Para empezar a trabajar con las consultas tendría que entrar a eXide – Xquery IDE



Esto no lo encuentro por el temario, ni como sigue a partir de aquí.



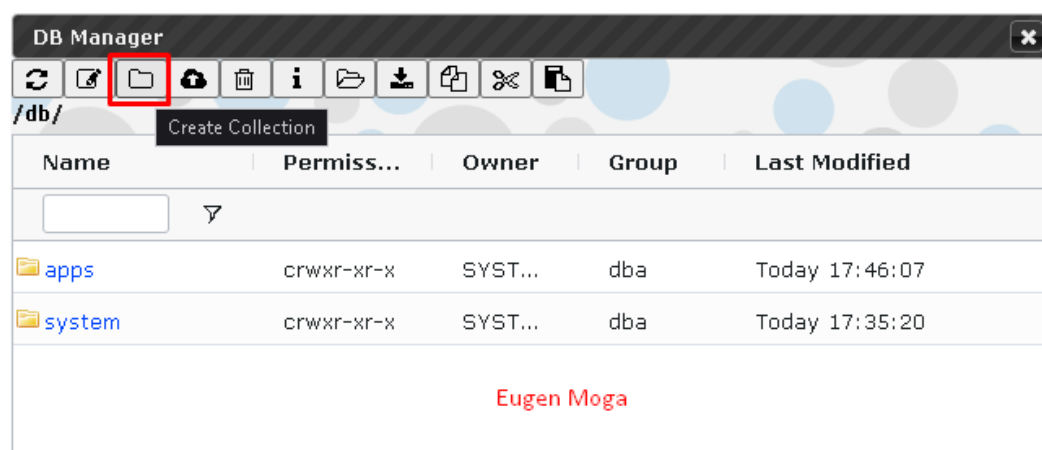
Según he visto por interne la forma de trabajar es cargar los archivos xml a los que voy a realizar las consultas. Para ello voy a File > Manage



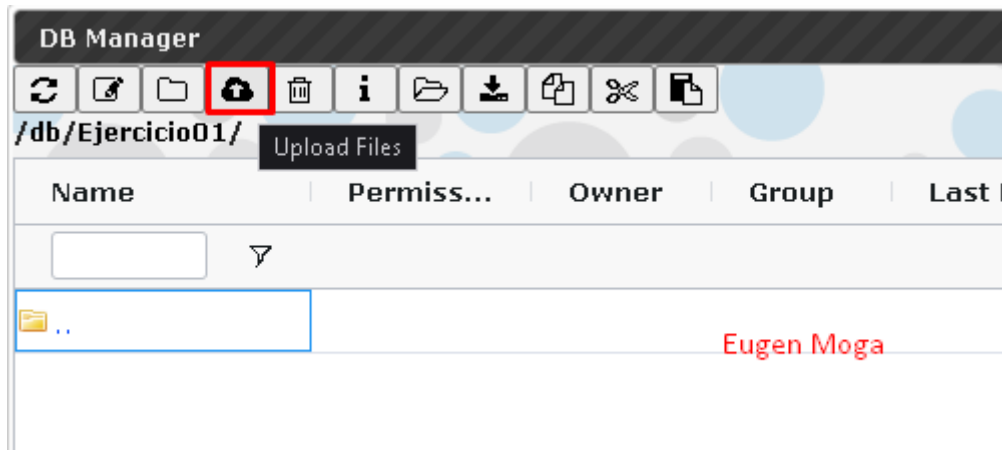
Me solicita iniciar sesión y pongo el usuario admin y la contraseña configurada en la instalación

The screenshot shows a 'Login' dialog box with a close button (X). It contains fields for 'User:' (with 'admin' entered) and 'Password:' (with masked characters). There is a checkbox labeled 'Remember me for 2 weeks' which is checked. At the bottom, there are 'Login' and 'Cancel' buttons.

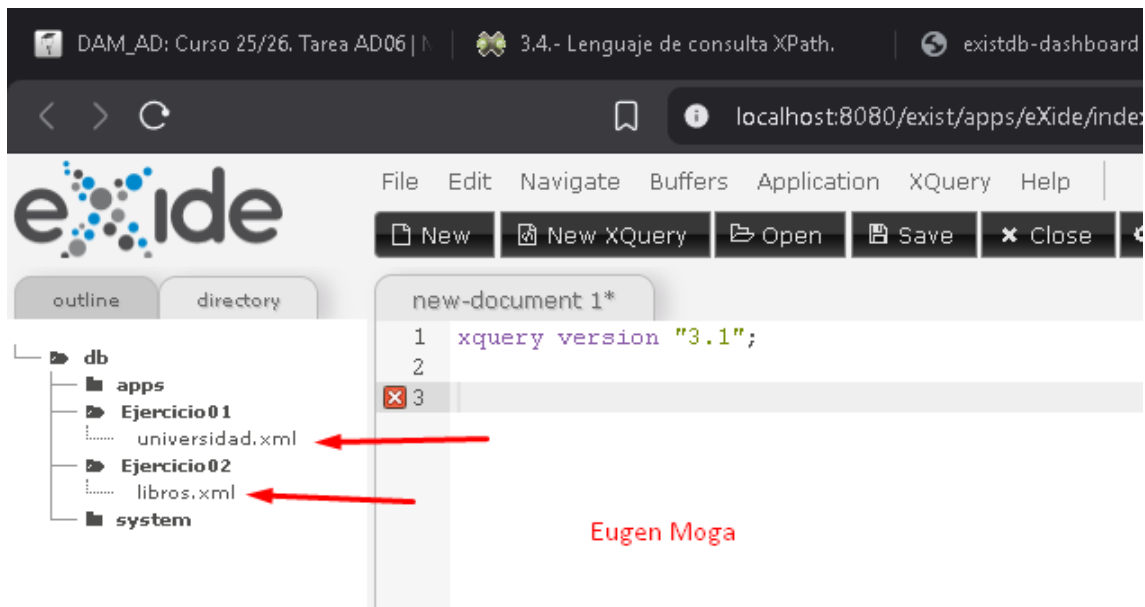
Voy a crear una carpeta para el ejercicio 1 y otra para el ejercicio 2 pulsando en el botón de la carpeta



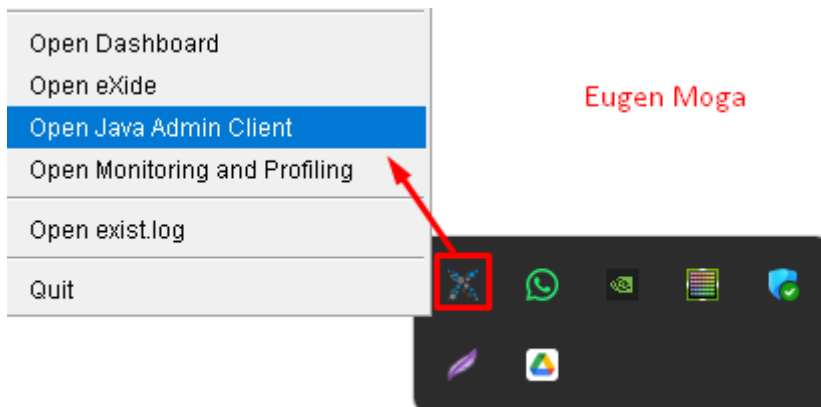
Ahora abro la carpeta Ejercicio01 y le pulso al botón para cargar archivos y cargo el archivo universidad.xml



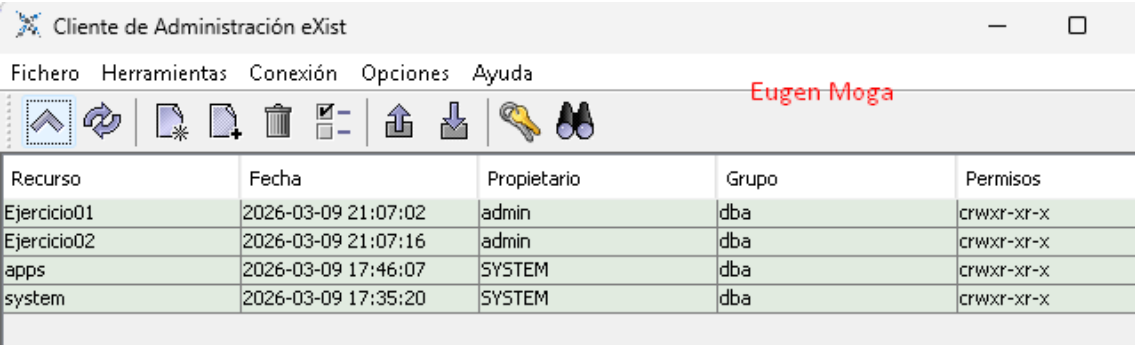
Hago lo mismo para la carpeta Ejercicio02 y cargo el archivo libros.xml



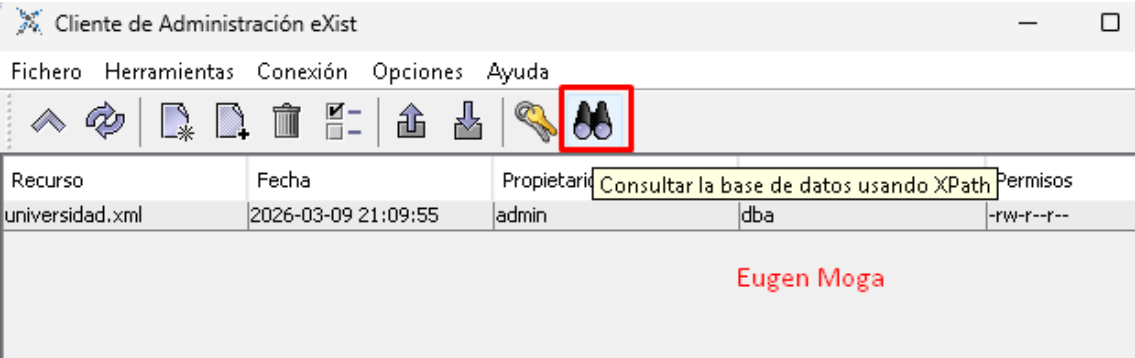
Veo que tambien se puede abrir el Cliente administración de eXist



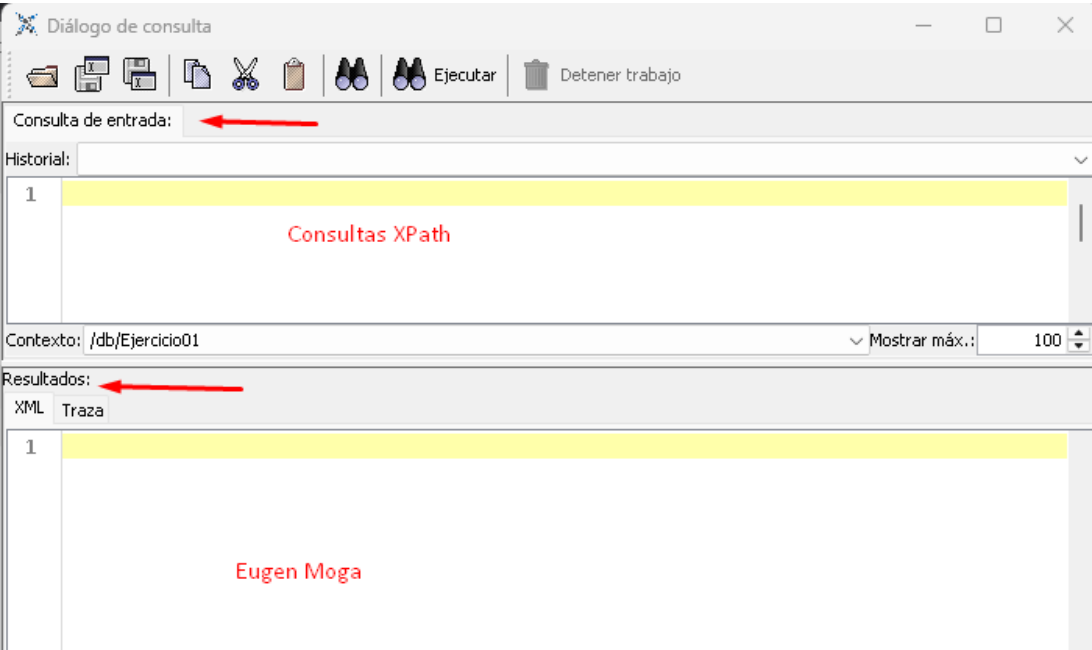
Esto me abre la aplicación y puedo ver las carpetas que he creado desde el navegador



Abro la carpeta Ejercicio01 y puedo ver el archivo universidad.xml Para ejecutar las consultas le pulso en el botón Consultar base de datos usando XPath



Esto me abre Dialogo de consulta y aquí es donde realizo las consultas y veo el resultado tras ejecutar



Ejercicio 1 XPATH (universidad.xml)

A continuación, paso a mostrar las consultas.

1. Nombre de la Universidad.

The screenshot shows the 'Diálogo de consulta' window. The 'Consulta de entrada:' field is empty. The 'Historial:' section shows a single query: `/universidad/nombre`. The 'Contexto:' is `/db/Ejercicio01`. The 'Resultados:' section shows the XML output: `<nombre>Universidad de Victoria</nombre>`. The 'Ejecutar' button is highlighted with a red box, and a red arrow points from the query in the history to the button. Another red arrow points from the button to the result.

Diálogo de consulta

Consulta de entrada:

Historial:

1 `/universidad/nombre`

Contexto: `/db/Ejercicio01`

Resultados:

XML Traza

1 `<nombre>Universidad de Victoria</nombre>`

2

Eugen Moga

2. Nombres de todas las asignaturas.

The screenshot shows the 'Diálogo de consulta' window. The 'Consulta de entrada:' field is empty. The 'Historial:' section shows a single query: `/universidad/asignaturas/asignatura/nombre`. The 'Contexto:' is `/db/Ejercicio01`. The 'Resultados:' section shows the XML output: `<nombre>Ofimática</nombre>`, `<nombre>Ingeniería del Software</nombre>`, `<nombre>Administración de Empresas</nombre>`, `<nombre>Derecho Internacional</nombre>`, `<nombre>Pedagogía</nombre>`, `<nombre>Didáctica</nombre>`, `<nombre>Tecnología de los Alimentos</nombre>`, `<nombre>Bases de Datos</nombre>`, and `<nombre>Historia del Pensamiento</nombre>`. The 'Mostrar más' button is visible. The 'Ejecutar' button is highlighted with a red box, and a red arrow points from the query in the history to the button. Another red arrow points from the button to the result.

Consulta de entrada:

Historial:

1 `/universidad/asignaturas/asignatura/nombre`

Contexto: `/db/Ejercicio01` Mostrar más

Resultados:

XML Traza

1 `<nombre>Ofimática</nombre>`

2 `<nombre>Ingeniería del Software</nombre>`

3 `<nombre>Administración de Empresas</nombre>`

4 `<nombre>Derecho Internacional</nombre>`

5 `<nombre>Pedagogía</nombre>`

6 `<nombre>Didáctica</nombre>`

7 `<nombre>Tecnología de los Alimentos</nombre>`

8 `<nombre>Bases de Datos</nombre>`

9 `<nombre>Historia del Pensamiento</nombre>`

10

Eugen Moga

3. Datos de las carreras cuyo id son c01 o c03.

Historial:	
1	<code>//carrera[@id = ('c01', 'c03')]</code>
Contexto: /db/Ejercicio01	
Resultados:	
XML	Traza
1	<code><carrera id="c01"></code>
2	<code> <nombre>I.T. Informática</nombre></code>
3	<code> <plan>2003</plan></code>
4	<code> <creditos>250</creditos></code>
5	<code> <centro>Escuela de Informática</centro></code>
6	<code></carrera></code>
7	<code><carrera id="c03"></code>
8	<code> <nombre>Dipl. Relaciones Laborales</nombre></code>
9	<code> <plan>2001</plan></code>
10	<code> <creditos>280</creditos></code>
11	<code> <centro>Facultad de Ciencias Sociales</centro></code>
12	<code> <subdirector>Alfonso Martín Luque</subdirector></code>
13	<code></carrera></code>
14	

Eugen Moga

4. Nombre de las asignaturas de primer trimestre.

Historial:	
1	<code>//asignatura[trimestre=1]/nombre</code>
2	
Contexto: /db/Ejercicio01	
Resultados:	
XML	Traza
1	<code><nombre>Ofimática</nombre></code>
2	<code><nombre>Administración de Empresas</nombre></code>
3	<code><nombre>Derecho Internacional</nombre></code>
4	<code><nombre>Bases de Datos</nombre></code>
5	

Eugen Moga

5. Nombre de la carrera que estudia el primer alumno.

Diálogo de consulta

Consultas de entrada:

Historial:

```
1 //carreras/carrera[@id = collection(/db)//alumnos/alumno[1]/
2 estudios/carrera/@codigo/xs:string()]/nombre
3
4
```

Contexto: /db/Ejercicio01

Resultados:

XML Traza

```
1 <nombre>I.T. Informática</nombre>
2
```

Eugen Moga

6. Apellido1 y Apellido2 de todas las mujeres.

Historial:

```
1 //alumno[sexo = 'Mujer']/concat(apellido1, ' ', apellido2)
2
```

Contexto: /db/Ejercicio01 Mostrar más.

Resultados:

XML Traza

```
1 Pérez García
2 Avalón Jiménez
3
```

Eugen Moga

7. Nombre de las asignaturas que estudia Víctor Manuel.

Diálogo de consulta

Consultas de entrada:

Historial:

```
1 //asignatura[@id = collection(/db)//alumno[nombre='Víctor Manuel']/
2 estudios/asignaturas/asignatura/@codigo]/nombre
3
4
5
```

Contexto: /db/Ejercicio01

Resultados:

XML Traza

```
1 <nombre>Ofimática</nombre>
2 <nombre>Administración de Empresas</nombre>
3 <nombre>Pedagogía</nombre>
4 <nombre>Historia del Pensamiento</nombre>
5
```

Eugen Moga

8. Primer apellido de los alumnos matriculados en Ofimática.

Consultas de entrada:

Historial:

```
1 //alumno[estudios/asignaturas/asignatura/@codigo = collection(/db)//asignatura
2 [nombre='Ofimática']/@id]/apellido1
3
4
5
```

Contexto: /db/Ejercicio01

Resultados:

XML Traza

```
1 <apellido1>Rivas</apellido1>
2 <apellido1>Pérez</apellido1>
3 <apellido1>Pérez</apellido1>
4 <apellido1>Avalón</apellido1>
5
```

Eugen Moga

9. Nombre de los alumnos matriculados en carreras que no tienen subdirector.

Consulta de entrada:

Historial:

```
1 //alumno[not(//carrera/@codigo=//carrera[subdirector]/@codigo)]/nombre
2
```

Contexto: /db/Ejercicio01 Mostrar

Resultados:

XML	Traza
1	<nombre>Victor Manuel</nombre>
2	<nombre>Luisa</nombre>
3	<nombre>Fernando</nombre>
4	<nombre>María</nombre>
5	

Eugen Moga

10. Nombre de los alumnos que estudian carreras cuyos planes son posteriores a 1999.

Consulta de entrada:

Historial:

```
1 //alumno[estudios/carrera/@codigo = collection(/db)//carrera[plan > 1999]]/@id/nombre
2
3
4
```

Contexto: /db/Ejercicio01 Mostrar máx.: 10

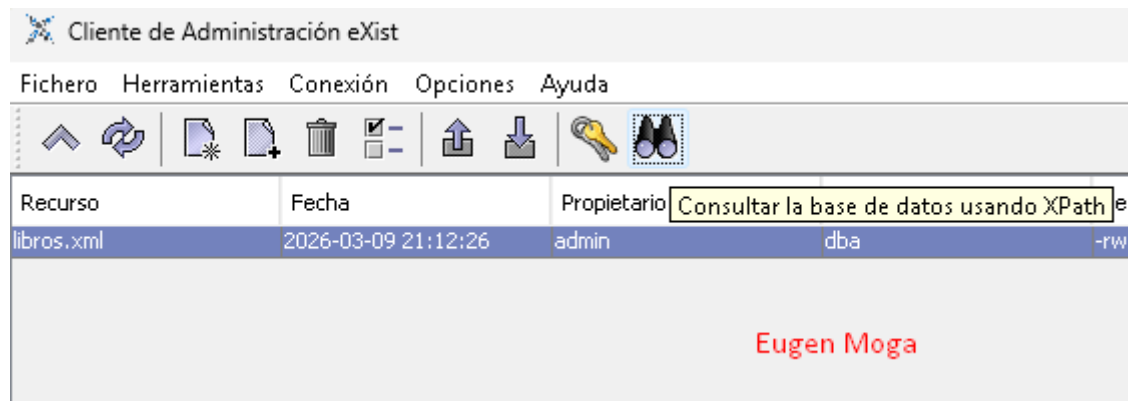
Resultados:

XML	Traza
1	<nombre>Victor Manuel</nombre>
2	<nombre>Luisa</nombre>
3	<nombre>Fernando</nombre>
4	<nombre>María</nombre>
5	

Eugen Moga

Ejercicio 2 XQUERY (libros.xml)

Para hacer las consultas Xquery desde el cliente administración eXist abro la carpeta Ejercicio02 que contiene el archivo libros.xml y le pulso en el prismático Consultar la base de datos.



Esto me abre la ventana de Dialogo de consulta donde voy a hacer la consulta y ver el resultado.



Procedo a realizar las consultas.

1. Listar todos los apellidos de los autores de libros.

Historial:

```
1 for $autor in /bib/libro/autor
2 return $autor/apellido
```

Contexto: /db/Ejercicio02

Resultados:

XML Traza

```
1 <apellido>Stevens</apellido>
2 <apellido>Stevens</apellido>
3 <apellido>Abiteboul</apellido>
4 <apellido>Buneman</apellido>
5 <apellido>Suciu</apellido>
6
```

Eugen Moga

2. Listar año y título de todos los libros, ordenados por el año.

Consulta de entrada:

Historial:

```
1 for $x in //libro
2 order by $x/@año
3 return
4   <libro>
5     {$x/@año} {$x/titulo}
6   </libro>
```

Contexto: /db/Ejercicio02

Resultados:

XML Traza

```
1 <libro año="1992"><titulo>Advan Programming for Unix environment</titulo></libro>
2 <libro año="1994"><titulo>TCP/IP Illustrated</titulo></libro>
3 <libro año="1999"><titulo> Economics of Technology for Digital TV</titulo></libro>
4 <libro año="2000"><titulo>Data on the Web</titulo></libro>
5
```

Eugen Moga

Mostrar máx.: 10

3. Listar los libros cuyo precio sea 65.95

Historial:	
1	for \$x in //libro[precio = 65.95]
2	return \$x
Contexto: /db/Ejercicio02	
Resultados:	
XML	Traza
1	<libro año="1994">
2	<titulo>TCP/IP Illustrated</titulo>
3	<autor>
4	<apellido>Stevens</apellido>
5	<nombre>W.</nombre>
6	</autor>
7	<editorial>Addison-Wesley</editorial>
8	<precio> 65.95</precio>
9	</libro>
10	<libro año="1992">
11	<titulo>Advan Programming for Unix environment</titulo>
12	<autor>
13	<apellido>Stevens</apellido>
14	<nombre>W.</nombre>
15	</autor>
16	<editorial>Addison-Wesley</editorial>
17	<precio>65.95</precio>
18	</libro>
19	

Eugen Moga

4. Listar los libros publicados después del año 1992.

Historial:	
1	for \$x in //libro[@año > 1992]
2	return \$x
Contexto: /db/Ejercicio02	
Resultados:	
XML	Traza
1	<libro año="1994">
2	<titulo>TCP/IP Illustrated</titulo>
3	<autor>
4	<apellido>Stevens</apellido>
5	<nombre>W.</nombre>
6	</autor>
7	<editorial>Addison-Wesley</editorial>
8	<precio> 65.95</precio>
9	</libro>
10	<libro año="2000">
11	<titulo>Data on the Web</titulo>
12	<autor>
13	<apellido>Abiteboul</apellido>
14	<nombre>Serge</nombre>
15	</autor>
16	<autor>
17	<apellido>Buneman</apellido>
18	<nombre>Peter</nombre>
19	</autor>
20	<autor>
21	<apellido>Suciu</apellido>
22	<nombre>Dan</nombre>
23	</autor>
24	<editorial>Morgan Kaufmann editorials</editorial>
25	<precio>39.95</precio>
26	</libro>
27	<libro año="1999">
28	<titulo> Economics of Technology for Digital TV</titulo>
29	<editor>
30	<apellido>Gerbarg</apellido>
31	<nombre>Darcy</nombre>
32	<afiliacion>CITI</afiliacion>
33	</editor>
34	<editorial>Kluwer Academic editorials</editorial>
35	<precio>129.95</precio>
36	</libro>
37	Eugen Moga

5. Listar año y título de los libros publicados por Addison-Wesley después del año 1992.

Historial:	
1	for \$x in //libro
2	where \$x/editorial = 'Addison-Wesley' and \$x/@año > 1992
3	return
4	<libro>
5	{ \$x/@año }
6	{ \$x/titulo }
7	</libro>

Contexto: /db/Ejercicio02

Resultados:	
Eugen Moga	
XML	Traza
1	<libro año="1994"><titulo>TCP/IP Illustrated</titulo></libro>
2	

6. Listar año y título de los libros que tienen más de un autor.

Historial:	
1	for \$x in //libro
2	where count (\$x/autor) > 1
3	return
4	<libro>
5	{ \$x/@año }
6	{ \$x/titulo }
7	</libro>

Contexto: /db/Ejercicio02

Resultados:	
Eugen Moga	
XML	Traza
1	<libro año="2000"><titulo>Data on the Web</titulo></libro>
2	

7. Listar año y título de los libros que no tienen autor.

Historial:	
1	for \$x in //libro
2	where count (\$x/autor) = 0
3	return
4	<libro>
5	{ \$x/@año }
6	{ \$x/titulo }
7	</libro>

Contexto: /db/Ejercicio02 Mostrar más

Resultados:	
Eugen Moga	
XML	Traza
1	<libro año="1999"><titulo>Economics of Technology for Digital TV</titulo></libro>
2	

8. Mostrar los apellidos de los autores que aparecen en el documento, sin repeticiones, ordenados alfabéticamente.

Historial:

```

1 for $x in distinct-values (//libro/autor/apellido)
2 order by $x
3 return $x
4

```

Contexto: /db/Ejercicio02

Resultados:

XML Traza

1	Abiteboul
2	Buneman
3	Stevens
4	Suciu
5	

Eugen Moga

9. Por cada libro, listar agrupado en un elemento <result> su título y autores

Historial:

```

1 for $x in //libro
2 return
3   <result>
4     {$x/titulo}
5     {$x/autor}
6   </result>
7

```

Contexto: /db/Ejercicio02 Mostr

Resultados:

XML Traza

```

1 <result><titulo>TCP/IP Illustrated</titulo><autor>
2 <apellido>Stevens</apellido>
3 <nombre>W.</nombre>
4 </autor></result>
5 <result><titulo>Advan Programming for Unix environment</titulo><autor>
6 <apellido>Stevens</apellido>
7 <nombre>W.</nombre>
8 </autor></result>
9 <result><titulo>Data on the Web</titulo><autor>
10 <apellido>Abiteboul</apellido>
11 <nombre>Serge</nombre>
12 </autor><autor>
13 <apellido>Buneman</apellido>
14 <nombre>Peter</nombre>
15 </autor><autor>
16 <apellido>Suciu</apellido>
17 <nombre>Dan</nombre>
18 </autor></result>
19 <result><titulo> Economics of Technology for Digital TV</titulo></result>
20

```

Eugen Moga

10. Por cada libro, obtener su título y el número de autores, agrupados en un elemento <libro>

Historial:

```
1 for $x in //libro
2   return
3     <libro>
4       {$x/titulo}
5       <num_autores>
6         {count($x/autor)}
7     </num_autores>
8   </libro>
9
```

Contexto: /db/Ejercicio02

Mostrar máx.: 100

Resultados:

XML

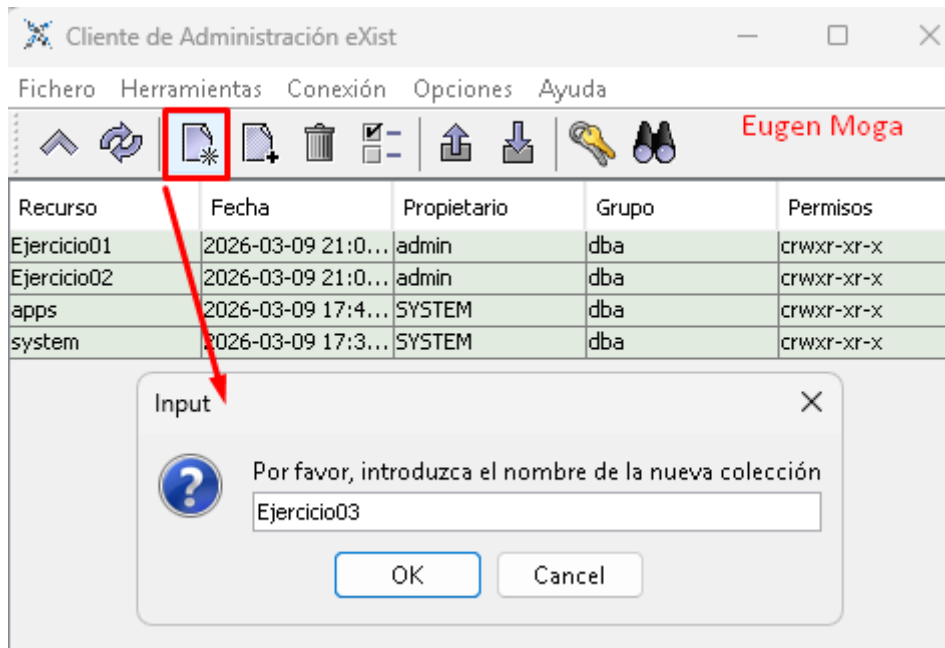
Traza

```
1 <libro><titulo>TCP/IP Illustrated</titulo><num_autores>1</num_autores></libro>
2 <libro><titulo>Advan Programming for Unix environment</titulo><num_autores>1</num_autores></libro>
3 <libro><titulo>Data on the Web</titulo><num_autores>3</num_autores></libro>
4 <libro><titulo> Economics of Technology for Digital TV</titulo><num_autores>0</num_autores></libro>
5
```

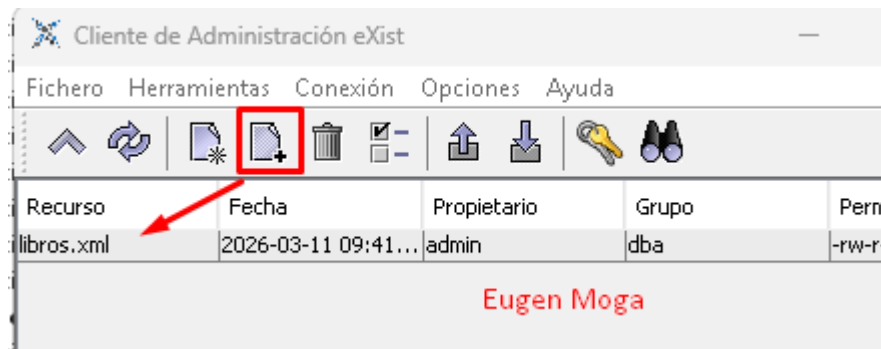
Eugen Moga

EJERCICIO 3. (utiliza eXist en un programa JAVA).

Para este ejercicio empiezo en el Cliente de Administración eXist creando una nueva colección y le pongo el nombre Ejercicio03



Abro la colección Ejercicio03 y subo el archivo libros.xml que será el archivo que voy a utilizar para hacer la consulta



Creo el proyecto AD06_ejercicio3 y dentro creo una clase BaseDatos_eXist.java que contendrá el método de acceso a la base de datos, método para ejecutar una consulta a la base de datos y un método para cerrar la conexión con la base de datos.

Clase BaseDatos_eXist.java

Dentro de esta clase empiezo explicando las clases que necesito importar para el correcto funcionamiento de la API XML

Necesito las clases básicas de acceso a bases de datos XML y las obtengo de api.base*

De api.modules* necesito los servicios adicionales como consultas XPath

De api.* necesito las clases principales de la API XML:DB

```
6
7 // Importo las clases necesarias de
8 import org.xmlldb.api.base.*;
9 import org.xmlldb.api.modules.*;
10 import org.xmlldb.api.*;
```

Dentro de la clase declaro las variables que necesito para configurar la conexión.

```
18 public class BaseDatos_eXist {
19     // Configuración de conexión
20     private static final String URI = "xmlldb:exist://localhost:8080/exist/xmlrpc";
21     private static final String COLECCION = "/db/Ejercicio03";
22     private static final String USUARIO = "admin";
23     private static final String PASS = "admin";
24 }
```

Ojo muy importante para que funcione la variable COLECCIÓN debe coincidir con la ruta donde tenemos cargado libros.xml en la base de datos. Esto se puede ver desde el Cliente de Administración eXist

```
type help or ? for help.
exist:/db> mkcol "Ejercicio03"
created collection.
exist:/db> cd "Ejercicio03"
exist:/db/Ejercicio03> 
```

Ahora creo un objeto Collection que representa la colección XML dentro de eXist

```
24
25 // Objeto que representa la coleccion XML dentro de eXist
26 private Collection coleccion;
27
```

Con XPathQueryService creo un servicio para poder ejecutar consultas XPath

```
27
28 // Servicio que permite ejecutar co
29 private XPathQueryService servicio;
```

Creo el constructor de la clase de momento sin inicializar la conexión ni tampoco el servicio de consulta.

```
31 // Constructor de la clase
32 public BaseDatos_eXist(){
33
34     // Inicialmente no hay co
35     this.coleccion = null;
36
37     // El servicio de consult
38     this.servicio = null;
39 }
```

Creo el método para inicializar el driver donde creo una instancia del driver de eXist-db

```
41 // Inicializo el driver de eXist-db
42 public void inicializarDriver() throws ClassNotFoundException, XMLDBException{
43     System.out.println("Inicializando driver de eXist-db.");
44
45     // Se crea una instancia del driver específico de eXist-db
46     Database database = new org.exist.xmldb.DatabaseImpl();
47 }
48
```


Creo el método conectar() para establecer la conexión con la base de datos. Y dentro de un bloque try-catch para tratar las excepciones. Con Database cargo la clase del driver y con DabtabaseManager registro el driver en el DatabaseManager para que java pueda conectarse a la base de datos de eXist.

```

48
49 // Establezco conexion con la base de datos
50 public void conectar() throws XMLDBException {
51     System.out.println("Conectando a: " + URI);
52
53     // Registrar el driver (IMPORTANTE)
54     try {
55         // Se carga dinámicamente la clase del driver
56         Database database = (Database) Class.forName("org.exist.xmldb.DatabaseImpl").newInstance();
57
58         // Se registra el driver en el DatabaseManager para que Java pueda conectarse a eXist-db
59         DatabaseManager.registerDatabase(database);
60     } catch (Exception e) {
61         // Si ocurre un error se lanza una excepción de base de datos XML
62         throw new XMLDBException();
63     }

```

Configuro las propiedades de conexión y le indico que para user le asigno la variable USUARIO y para password le asigno la contraseña que la tengo guardada en la variable PASS

```

64
65 // Primero configurar las propiedades de conexión
66 java.util.Properties props = new java.util.Properties();
67 props.setProperty("user", USUARIO);
68 props.setProperty("password", PASS);
69

```

Obtengo la colección con autenticación y se la paso al objeto colección. Y hago una comprobación por si la colección no existe o no se puede acceder que muestre el error de XMLDBException.

```

69
70 // Obtener la colección con autenticación
71 coleccion = DatabaseManager.getCollection(URI + COLECCION, USUARIO, PASS);
72
73 // Si la colección no existe o no se puede acceder
74 if (coleccion == null) {
75
76     // Se lanza un error indicando que la colección no existe
77     throw new XMLDBException(ErrorCodes.NO_SUCH_COLLECTION,
78         "No se puede acceder a la colección: " + COLECCION);
79 }

```

Con XPathQueryService obtengo el servicio para realizar la consulta y si todo es satisfactorio muestro el mensaje que se ha conectado correctamente con el usuario.

```
80  
81  
82 // Se obtiene el servicio para realizar consultas XPath  
83 servicio = (XPathQueryService) coleccion.getService("XPathQueryService", "1.0");  
84 System.out.println("Conectado correctamente con el usuario: " + USUARIO);  
}
```

Creo el método ejecutarConsulta de tipo ResultSet y devuelvo el resultado.

```
86 // Método para ejecutar consultas XPath  
87 public ResultSet ejecutarConsulta(String consulta) throws XMLDBException{  
88     System.out.println("\nEjecutando consulta: " + consulta);  
89  
90     // Ejecuta la consulta XPath sobre la colección  
91     ResultSet resultado = servicio.query(consulta);  
92  
93     // Devuelve el conjunto de resultados  
94     return resultado;  
95 }
```

Por último, creo el método para cerrar la conexión con la base de datos.

```
97 // Método para cerrar la conexión con la base de datos  
98 public void cerrarConexion() throws XMLDBException{  
99     if (coleccion != null){  
100         coleccion.close();  
101         System.out.println("Conexion cerrada correctamente.");  
102     }  
103 }
```

Clase AD06_ejercicio3.java

Esta es la clase principal y la que tiene el método main. Antes de empezar voy a enumerar las importaciones que tengo que hacer para que funcione correctamente. Estas clases son de la API XML:DB y se importan todas desde org.xmldb.api.base.

Resource: representa un recurso XML devuelto por una consulta.

ResourceIterator: permite recorrer los resultados de una consulta.

ResourceSet: es el conjunto de resultados devueltos por una consulta

XMLDBException: trata las excepciones de los errores de una base de datos XML

```
6
7 // Importo las clases necesarias de la API XML
8 import org.xmldb.api.base.Resource;
9 import org.xmldb.api.base.ResourceIterator;
10 import org.xmldb.api.base.ResourceSet;
11 import org.xmldb.api.base.XMLDBException;
12
```

Dentro del método main creo un objeto de la propia clase para ejecutar el programa y llamo al método ejecutar().

```
19 public class AD06_ejercicio3 {
20     /**
21      * Metodo principal del programa
22      *
23      * @param args the command line arguments
24      */
25     public static void main(String[] args) throws ClassNotFoundException {
26
27         // Se crea un objeto de la propia clase para ejecutar el programa
28         AD06_ejercicio3 programa = new AD06_ejercicio3();
29
30         // Llamo al método que contiene la lógica principal
31         programa.ejecutar();
32
33     }
```

Creo el método ejecutar() donde empiezo creando un objeto de la clase que gestiona la conexión con la base de datos.

```
35 // Metodo para ejecutar programa
36 public void ejecutar() throws ClassNotFoundException{
37
38     // Se crea un objeto de la clase que gestiona la c
39     BaseDatos_eXist db = new BaseDatos_eXist();
40
```

Dentro de un bloque try-catch empiezo llamando al método para inicializar el driver de la base de datos. Y después llamo al método para conectar con la base de datos.

```
41 try {
42     // Inicializa el driver
43     db.inicializarDriver();
44
45     // Establece la conexión
46     db.conectar();
47
```

Creo un variable consulta de tipo String y le paso la consulta. Con ResultSet ejecuto la consulta.

```
48 // Consulta XPath que obtiene todos los títulos de los
49 String consulta = "//libro/titulo";
50
51 // Se ejecuta la consulta en la base de datos
52 ResultSet resultado = db.ejecutarConsulta(consulta);
```

Muestro en pantalla el mensaje Listado de Libros y con ResourceIterator obtengo un iterador para recorrer los resultados.

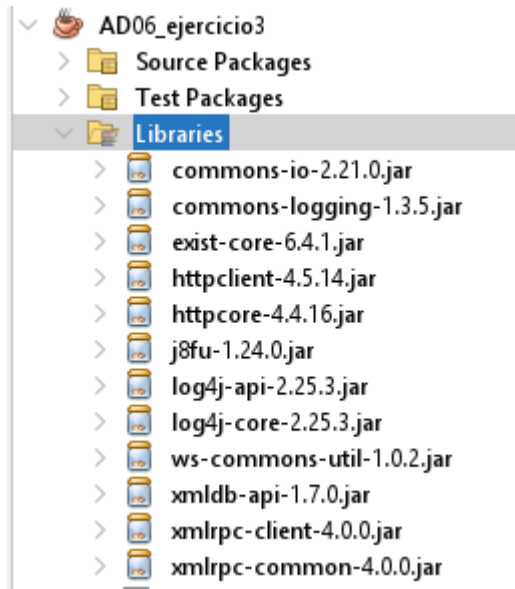
Dentro de un bucle while itero mientras existas más resultados e imprimo el resultado en pantalla con System.out.println.

Cierro el bloque catch y trato las excepciones de XMLDBException en caso de error al conectar con la base de datos.

```
54      System.out.println("          LISTADO DE LIBROS          ");
55
56      // Se obtiene un iterador para recorrer los resultados
57      ResourceIterator iterador = resultado.getIterator();
58
59      // Mientras existan más resultados
60      while (iterador.hasMoreResources()){
61          Resource recurso = iterador.nextResource();
62          System.out.println(recurso.getContent().toString());
63      }
64
65      }catch (XMLDBException e){
66          System.err.println("Error al conectar con eXist-db " + e.getMessage());
67          e.printStackTrace();
68      }
```

Conectores

Paso a detallar los conectores que he necesitado. Al instalar eXist en la carpeta de instalación C:\eXist-db\lib están todos los conectores y he ido pasando al proyecto de netbeans los que necesitaba en función de los errores que me salían (un total de 12 conectores)



commons-io-2.21.0.jar Librería de utilidades para manejo de entrada/salida (I/O) en Java: lectura, escritura y manipulación de archivos.

commons-logging-1.3.5.jar Proporciona una API de logging genérica que permite a las librerías usar distintos sistemas de registro (Log4j, Java Logging, etc.).

exist-core-6.4.1.jar Núcleo principal de eXist-db, contiene las clases que permiten interactuar con la base de datos XML.

httpclient-4.5.14.jar Librería para realizar peticiones HTTP desde Java, usada por el cliente para comunicarse con el servidor eXist.

httpcore-4.4.16.jar Implementa los componentes básicos del protocolo HTTP utilizados por httpclient.

j8fu-1.24.0.jar Librería auxiliar utilizada por eXist que proporciona estructuras funcionales y utilidades Java (como tuplas).

log4j-api-2.25.3.jar API del sistema Log4j, usada para generar mensajes de registro (logs) en aplicaciones Java.

log4j-core-2.25.3.jar Implementación principal de Log4j, encargada de procesar y almacenar los logs generados por la aplicación.

ws-commons-util-1.0.2.jar Librería de utilidades usada por Apache XML-RPC para manejar datos y comunicación web.

xmldb-api-1.7.0.jar API estándar XML:DB que define las interfaces para acceder a bases de datos XML como eXist.

xmlrpc-client-4.0.0.jar Implementa el cliente XML-RPC que permite a Java enviar consultas al servidor eXist.

xmlrpc-common-4.0.0.jar Contiene clases comunes usadas por las librerías XML-RPC cliente y servidor.

Funcionamiento del programa

Al ejecutar el programa puedo ver que funciona correctamente mostrando los mensajes haciendo la consulta y mostrando los libros del documento libros.xml

```
Output - AD06_ejercicio3 (run)
run:
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8 -Dsun.jnu.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8
Iniciando driver de eXist-db.
Conectando a: xmldb:exist://localhost:8080/exist/xmlrpc
Conectado correctamente con el usuario: admin
Ejecutando consulta: //libro/titulo
LISTADO DE LIBROS
<titulo>TCP/IP Illustrated</titulo>
<titulo>Advan Programming for Unix environment</titulo>
<titulo>Data on the Web</titulo>
<titulo> Economics of Technology for Digital TV</titulo>
BUILD SUCCESSFUL (total time: 2 seconds)
```

Eugen Moga

Ojo para que esto funcione correctamente hay que tener instalado la base de datos eXist cargar el archivo libros.xml en una colección en mi caso la colección esta en: /db/Ejercicio03 si el archivo xml está en otra colección hay que cambiar la variable COLECCIÓN de la case BaseDatos_eXist

```
18 public class BaseDatos_eXist {
19     // Configuración de conexión
20     private static final String URI = "xmldb:exist://localhost:8080/exist/xmlrpc";
21     private static final String COLECCION = "/db/Ejercicio03";
```

Pasa lo mismo con el usuario y la contraseña, yo utilizo el usuario: admin con la contraseña: admin Si tienes otro usuario y contraseña tendrás que cambiarlas en las variables correspondientes en la clase BaseDatos_eXist

```
22  → private static final String USUARIO = "admin";  
23  → private static final String PASS = "admin";  
24
```